

# 第 9 章：Reasoning Model、RLVR 与过程监督

## 本章符号说明

| 符号/缩写             | English                                        | 中文          | 含义                            |
|-------------------|------------------------------------------------|-------------|-------------------------------|
| Reasoning Model   | Reasoning Model                                | 推理模型        | 会在回答前分配额外推理计算的模型。             |
| Test-time Compute | Test-time Compute                              | 测试时计算       | 推理阶段额外使用的 token、采样、搜索或工具调用预算。 |
| CoT               | Chain-of-Thought                               | 思维链         | 中间推理过程。                       |
| Hidden CoT        | Hidden Chain-of-Thought                        | 隐藏思维链       | 不直接展示给用户的内部推理轨迹。              |
| RLVR              | Reinforcement Learning with Verifiable Rewards | 可验证奖励强化学习   | 用可自动验证的 reward 训练模型。          |
| PRM               | Process Reward Model                           | 过程奖励模型      | 对中间步骤打分。                      |
| ORM               | Outcome Reward Model                           | 结果奖励模型      | 对最终答案或最终状态打分。                 |
| GRPO              | Group Relative Policy Optimization             | 组相对策略优化     | 用组内相对 reward 训练策略的 RL 算法。     |
| Best-of-N         | Best-of-N Sampling                             | N 选 1 采样    | 采样多个答案，再用 verifier 选最优。       |
| Self-Consistency  | Self-Consistency                               | 自一致性        | 多条推理路径投票或聚合。                  |
| Verifier          | Verifier                                       | 验证器         | 判断答案、步骤、代码或状态是否正确。            |
| MCTS              | Monte Carlo Tree Search                        | 蒙特卡洛树搜索     | 通过搜索树探索候选推理路径。                |
| $x$               | Prompt                                         | 输入问题        | 用户问题或任务。                      |
| $y$               | Response                                       | 模型回答        | 模型生成的完整回答或轨迹。                 |
| $y_t$             | Reasoning Step                                 | 第 $t$ 个推理步骤 | 推理轨迹中的一步。                     |
| $R(x,y)$          | Reward                                         | 奖励          | 对回答或轨迹的得分。                    |

## 本章目标

- 理解 reasoning model 和普通 instruction model 的区别。
- 能解释为什么 test-time compute 可以提升复杂任务表现。
- 掌握 PRM、ORM、verifier、best-of-n、self-consistency 的关系。
- 能讲清 DeepSeek-R1、OpenAI o-series、Qwen3 等报告里常见的 RLVR / thinking mode。
- 理解 reasoning 带来的安全和评测新问题。

## Reasoning Model 是什么

普通 chat model 倾向于直接生成答案。Reasoning model 会在复杂任务上使用更多内部推理预算、采样或工具调用，再输出最终答案。

核心变化：

- 训练目标从“像高质量答案”变成“能通过验证和测试”。
- 推理阶段从单次生成变成可搜索、可验证、可反思。
- 对数学、代码、科学、工具调用、视觉推理更有效。

面试表达：

Reasoning model 的本质不是神秘的“会思考”，而是训练和推理范式变化：post-training 中用可验证奖励强化长推理轨迹，inference 时给更多 compute 让模型搜索、检查和修正。

## Test-time Compute

Test-time compute 包括：

| 类型               | English            | 中文    | 例子                                 |
|------------------|--------------------|-------|------------------------------------|
| Longer Thinking  | Longer Thinking    | 更长推理  | 允许更多 hidden reasoning tokens。      |
| Best-of-N        | Best-of-N Sampling | 多采样择优 | 采样多个答案，用 verifier 选最好。             |
| Self-Consistency | Self-Consistency   | 自一致性  | 多条链投票。                             |
| Tool Execution   | Tool Execution     | 工具执行  | Python、browser、compiler、unit test。 |
| Search           | Search             | 搜索    | tree search、beam、planner-executor。 |

关键直觉：

- 对不可验证任务，更多推理可能只是更长的幻觉。
- 对可验证任务，更多推理可以带来“生成-验证-修正”的收益。
- 现代 API 常提供 reasoning effort / thinking budget，本质就是控制质量、延迟、成本的旋钮。

## PRM、ORM 与 Verifier

ORM 只看最终结果：

$$R_{ORM}(x,y)=I[final(y)=target]$$

PRM 看中间步骤：

$$R_{PRM}(x,y_{1:t}) \rightarrow \{0,1\} \text{ or } \mathbb{R}$$

Verifier 更泛化，可以检查：

- 数学最终答案。
- 代码单元测试。
- 工具调用最终数据库状态。
- RAG 引用是否支持答案。
- agent 是否完成用户目标。

面试回答：

ORM 信号简单但稀疏，PRM 信号密集但贵且可能奖励表面推理。Verifier 可以用于训练，也可以用于 inference-time reranking。现代 reasoning 系统常把它们组合起来。

## RLVR 训练闭环

RLVR 典型流程：

1. 给模型一个问题  $x$ 。
2. 模型采样多个回答或轨迹  $y_1, \dots, y_n$ 。
3. 外部 verifier 给 reward。
4. 用 RL 更新模型，使高 reward 轨迹概率升高。

可以写成：

$$\max_{\theta} \mathbb{E}_{y \sim \pi_{\theta}(\cdot | x)} [R(x,y)]$$

为什么它有效：

- 数学和代码天然有 reward。
- agent 可以用最终状态做 reward。
- 模型能从失败样本中学习策略。
- 相比人工偏好，更可扩展。

为什么它危险：

- reward 写错会被 exploit。
- benchmark 泄漏会导致虚高。

- 模型可能学会 shortcut，而不是学会真实能力。
- 长 CoT 可能变成不可审计的内部策略。

## GRPO 的直觉

GRPO 给同一个问题采样一组回答，用组内相对表现构造 advantage：

$$A_i = r_i - \bar{r} / \sigma_r$$

其中：

$$\bar{r} = 1 / G \sum_{j=1}^G r_j$$

优点：

- 不需要 value model。
- 显存更省。
- 很适合大模型上做 reasoning RL。

局限：

- 如果一组样本全对或全错，学习信号会弱。
- 对 reward 稀疏任务，采样策略很重要。
- 相对 reward 会鼓励组内胜出，不保证绝对质量。

## DeepSeek-R1 的训练启发

DeepSeek-R1 把 reasoning RL 推到开源社区视野里。核心启发：

- R1-Zero 显示大规模 RL 可以让 reasoning behavior 自发出现。
- 纯 RL 可能出现可读性差、语言混杂等问题。
- R1 用 cold-start data + 多阶段 RL 改善可读性和稳定性。
- distilled models 说明大模型 reasoning 轨迹可以迁移到小模型。

面试表达：

DeepSeek-R1 的意义不只是模型效果，而是证明了“可验证 reward + 大规模 RL”可以显著激发推理能力。但它也暴露出 reasoning trace 可读性、语言混杂、reward 设计和安全监控问题。

## Qwen3 的 Thinking / Non-thinking 统一

Qwen3 技术报告强调一个实用方向：同一个模型支持 thinking mode 和 non-thinking mode，并通过 thinking budget 控制推理深度。

面试可以这样讲：

这说明产业界不希望永远让用户手动切模型。更理想的系统是统一模型或路由器根据任务复杂度决定是否深度推理，在成本、延迟和质量之间动态折中。

## CoT 监控与隐藏推理

Reasoning model 带来一个新问题：如果模型的真实推理过程不展示，怎么监控安全？

需要区分：

- 给用户看的解释。
- 模型内部 hidden CoT。
- 系统用于安全审计的 trace。

风险：

- 展示完整 CoT 可能泄露隐私、安全策略或训练数据。
- 不展示 CoT 会降低可解释性。
- 模型可能出现 reasoning-output discrepancy，即内部想法和外部回答不一致。

面试表达：

CoT 不是越透明越好。产品上常展示摘要而不是原始推理；安全上需要内部监控、红队、行为 eval 和工具轨迹审计一起做。

## Reasoning Eval

常见评测：

| Benchmark      | English                                       | 中文       | 测什么            |
|----------------|-----------------------------------------------|----------|----------------|
| AIME           | American Invitational Mathematics Examination | 美国邀请数学竞赛 | 数学推理。          |
| GPQA           | Graduate-Level Google-Proof QA                | 研究生级问答   | 科学知识和推理。       |
| FrontierMath   | Frontier Math Benchmark                       | 前沿数学基准   | 难数学问题。         |
| SWE-bench      | Software Engineering Benchmark                | 软件工程基准   | 真实代码 issue 修复。 |
| Terminal-Bench | Terminal Benchmark                            | 终端任务基准   | 命令行多步任务。       |
| BrowseComp     | Browsing Competition Benchmark                | 浏览检索基准   | 隐蔽信息查找。        |

| Benchmark | English                   | 中文          | 测什么       |
|-----------|---------------------------|-------------|-----------|
| TAU-bench | Tool-Agent-User Benchmark | 工具-智能体-用户基准 | 多轮业务工具任务。 |

不要只报 benchmark 分数。要追问：

- 是否用了 tool?
- 是否用了 high reasoning effort?
- 是否用了多次采样?
- 是否有外部 scaffold?
- 任务集是否污染?
- 结果是 pass@1 还是 best-of-n?

## 本章面试高频问题

### 1. Reasoning model 和普通 LLM 的区别？

普通 LLM 主要直接生成答案。Reasoning model 通过 RL 和 inference-time compute 学会更长的搜索、验证和修正，尤其适合数学、代码、科学、工具调用等可验证任务。

### 2. 为什么 PRM 有用？

PRM 给中间步骤提供监督，可以减少只看最终答案时的 credit assignment 问题。但 PRM 标注贵，也可能被模型学成表面推理模板。

### 3. 为什么 RLVR 比 RLHF 更适合代码和数学？

代码和数学有自动验证器，比如单元测试、编译器、答案检查器。相比人类偏好，这些 reward 更便宜、更客观、更可扩展。

### 4. 为什么更多 thinking 不一定更好？

如果任务不可验证，模型可能只是生成更长、更自信的错误推理。更多 thinking 还会增加延迟、成本和安全审计难度。

### 5. 面试被问 DeepSeek-R1 怎么答？

答训练范式：R1-Zero 用大规模 RL 激发推理；R1 加 cold-start data 和多阶段训练改善可读性；核心贡献是证明 verifiable reward + RL 可以扩展 reasoning，但也带来 reward hacking、语言混杂、CoT 监控等问题。

## 本章 References

- [Let's Verify Step by Step](#)
- [DeepSeekMath: Pushing the Limits of Mathematical Reasoning](#)
- [DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning](#)
- [Qwen3 Technical Report](#)
- [OpenAI o3 and o4-mini System Card](#)
- [Deliberative Alignment: Reasoning Enables Safer Language Models](#)
- [Think Fast: Estimating No-CoT Task-Completion Time Horizons of Frontier AI Models](#)

[章节索引](#)

[← 上一章](#) [下一章 →](#)

---